

开发环境篇

目标平台：LicheePi 4A / TH1520 · 操作系统：RevyOS · 工具链：rui + GNU Toolchain

1.1 RISC-V 平台认知与 rui 开发环境安装

学习目标

- 区分 RISC-V ISA、处理器核、SoC、开发板、操作系统和开发工具链六个层次
- 识别 LicheePi 4A 的主要接口并说明实验用途
- 形成个人课程环境清单，确认硬件、线缆和主机软件
- 说明 `rui` 在 RISC-V 开发环境中的作用，完成安装与版本检查
- 更新软件源、查询可用工具链，记录所选工具链信息
- 使用编译器版本、目标三元组和简单编译结果验证工具链

平台分层：从 ISA 到应用

RISC-V 是开放指令集架构（ISA）——定义了处理器执行指令的接口。芯片和开发板是不同层次的概念。建立分层思维是后续排查「镜像与板卡不匹配」「工具链架构错误」等问题的基础。

RISC-V ISA（开放指令集架构规范）

↓

处理器核 / SoC（TH1520）

↓

开发板（LicheePi 4A）

↓

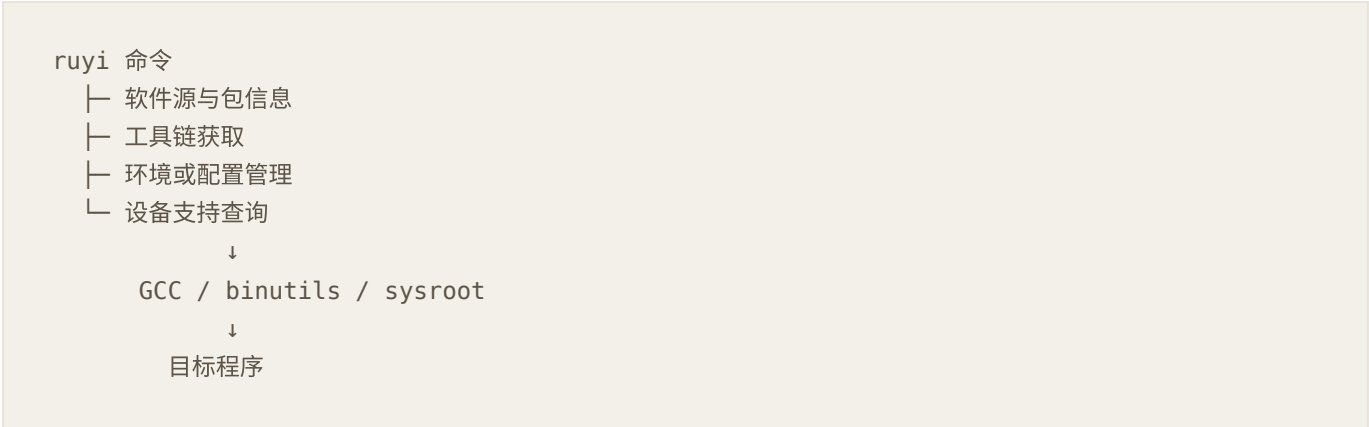
操作系统（RevyOS）

↓

开发工具（rui、编译器、SSH、串口工具）

ruyi 与工具链

`ruyi` 是 RuyiSDK 的包管理与环境管理入口。它负责发现和获取 RISC-V 开发所需的软件包与工具链——真正编译代码的是 GCC。两者关系类似「应用商店」与「应用程序」。



环境准备

项目	要求	检查方式
主机环境	Linux、macOS 或 Windows；课程命令以 Linux 为主	<code>uname -a</code>
目标平台	LicheePi 4A 与可用启动介质	核对板卡丝印和包装型号
连接工具	USB 数据线、USB-TTL 串口模块、网线或 Wi-Fi	确认线缆支持数据传输
网络	可访问 RuyiSDK 软件源	浏览器或连通性检查
基础工具	<code>curl</code> 或 <code>wget</code> 、文件校验工具	<code>command -v curl</code>

安装权限

用户目录可写

```
test -w "$HOME"
```

常见陷阱

- 部分 USB-C 线仅供电、无数据通道 → 无法识别设备时先换线
- 串口电平必须为 3.3V → 5V 可能损坏板卡
- 工具链需选 `linux-gnu` 而非 `unknown-elf`（裸机工具链）

操作步骤

步骤 1：记录主机环境

```
$ uname -a
$ uname -m
```

`uname -m` 确认主机架构。主机架构与目标板架构可以不同——这正是需要交叉编译的原因。

步骤 2：识别课程平台层次

层次	本课程对象
ISA	RISC-V 64 位
SoC	TH1520
开发板	LicheePi 4A
操作系统	RevyOS
工具入口	<code>ruyi</code> 、GNU Toolchain、SSH/SCP、串口终端

步骤 3：安装 `ruyi` 并确认命令位置

```
$ command -v ruyi
$ ruyi --version
```

第一条命令确认 shell 实际调用的文件路径，第二条记录版本号。若刚修改 `PATH`，可重新打开终端或执行 `hash -r`。

步骤 4：更新软件源并查询工具链

```
$ ruyi --help
# 根据帮助信息执行软件源更新与包搜索
$ ruyi update
$ ruyi list | grep toolchain
```

搜索关键词 `toolchain`、`gcc` 或 `riscv`。记录候选包名称、版本、目标架构和 ABI。

步骤 5：安装并验证编译器

```
$ riscv64-unknown-linux-gnu-gcc --version
$ riscv64-unknown-linux-gnu-gcc -dumpmachine
```

`-dumpmachine` 应显示 RISC-V 目标三元组。再编译最小程序验证：

```
int main(void) { return 0; }
```

使用 `file` 检查输出文件架构。正确验收方式：用 `file` 确认目标架构为 RISC-V ELF。在主机上 `./a.out` 会因架构不同而报错——这不是正确的验收方法。

课堂练习

1. 解释「LicheePi 4A 是 RISC-V」这句话为什么不够准确，并用 ISA、SoC、开发板三个术语重新表述。

2. 比较包含 `linux-gnu` 与包含 `unknown-elf` 的两个目标三元组，说明它们分别更适合 Linux 应用还是裸机程序。

本节成果

- 课程平台分层图
- LicheePi 4A 接口标注记录
- 个人软硬件环境清单
- `ruyi` 安装路径和版本记录
- 软件源更新与工具链查询记录
- 工具链目标三元组、版本和最小编译验证结果

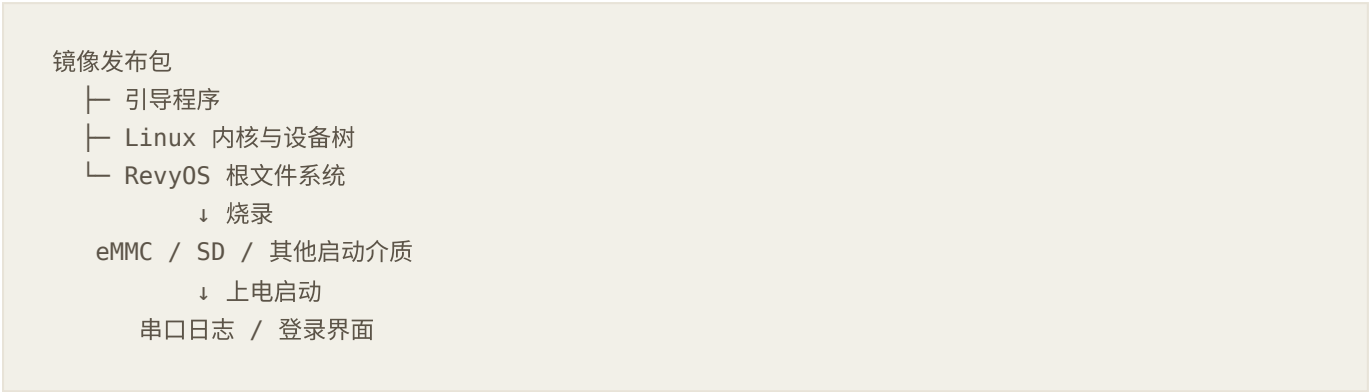
1.2 镜像烧录、首次启动与开发通道建立

学习目标

- 说明镜像、启动介质、引导程序和根文件系统之间的关系
- 在确认板卡型号、镜像版本和目标介质后完成安全烧录
- 首次启动后检查用户、网络、时间和 apt 软件源状态
- 说明串口登录与 SSH 登录的差异，为故障排查选择合适通道
- 完成 SSH 首次连接、主机指纹核对和密钥登录配置
- 使用 SCP 在主机与开发板之间双向传输文件并校验内容

镜像烧录：分区级数据写入

烧录工具将引导程序、内核、设备树和根文件系统按分区结构写入目标介质。错误的镜像或错误的目标设备可能导致无法启动，严重时覆盖主机磁盘。



三条通道，三个角色

通道	角色	依赖	适用场景
串口（UART）	本地调试 · 最后保障线	USB-TTL 3.3V	观察启动日志、修复网络配置
SSH	远程主工作通道	网络 + sshd	日常开发、远程命令执行

SCP

文件传输通道

复用 SSH 连
接

程序部署、日志下载

黄金组合：串口留着备用（断网时的救生索），SSH 做主通道，SCP 负责搬文件。

操作步骤

步骤 1：记录并校验镜像

```
$ sha256sum <image-file>
```

将计算结果与发布方提供的校验值比对。校验不一致时停止烧录，重新下载。

步骤 2：识别目标设备并执行烧录

一次只连接一块待烧录板，使用烧录工具的设备列表命令确认目标。严格使用该镜像版本的发布说明，不拼接其他板卡或其他年份的命令。保存完整日志，等待工具明确报告成功后再断电。

步骤 3：观察首次启动

首次启动可能进行文件系统扩展、密钥生成等初始化操作，耗时可能长于后续启动。请耐心等待，优先使用串口观察引导日志——屏幕无画面时串口仍可确认启动状态。

步骤 4：检查系统与账户

```
$ cat /etc/os-release      # 确认 RevyOS
$ uname -a                 # riscv64
$ id                       # 记录当前用户
$ hostnamectl              # 主机名
```

步骤 5：检查网络、时间和软件源

```
$ ip -br addr
$ ip route
$ timedatectl
$ grep -R --no-filename -h '^deb ' /etc/apt/sources.list /etc/apt/sources.list.d 2>/dev/null
$ sudo apt update
```

先检查链路、地址、路由和时间，再执行 `apt update`。系统时间严重错误时，TLS 证书和仓库元数据校验都会失败——`apt update` 报错应先排查时间而非立即换源。

步骤 6：建立串口连接

断电接线：GND-GND、主机模块 TX 到板端 RX、主机模块 RX 到板端 TX。接线前核对板卡资料的波特率和电平要求。

步骤 7：配置 SSH 密钥登录

```
# 板端启用 SSH
$ sudo systemctl enable --now ssh

# 主机生成密钥对
$ ssh-keygen -t ed25519 -C "riscv-course"

# 首次连接——核对主机指纹
$ ssh user@board-ip
# 看到指纹时通过串口或受信任渠道核对，确认无误后再输入 yes

# 部署公钥
$ ssh-copy-id user@board-ip

# 免密验证
$ ssh user@board-ip 'hostname; uname -m'
```

私钥留在主机，不复制给他人。公钥写入板端 `~/.ssh/authorized_keys`。

步骤 8：SCP 双向传输并校验

```
$ printf 'RISC-V course transfer test\n' > transfer-test.txt
$ sha256sum transfer-test.txt
$ scp transfer-test.txt user@board-ip:/tmp/
$ ssh user@board-ip 'sha256sum /tmp/transfer-test.txt'
```



```
# 从板端下载
$ scp user@board-ip:/etc/os-release ./board-os-release.txt
```

主机和板端哈希一致 = 传输无损坏。

课堂练习

1. 将「开发板无法启动」拆分为镜像、烧录、供电、启动介质和显示通道五类原因，每类写出一个验证方法。
2. 分别说明以下故障应优先使用串口还是 SSH：板端没有 IP、sshd 配置错误、系统启动失败、需要批量上传文件。

指纹变更警告：可能是开发板重装系统，也可能连错了设备。先核对 IP 和串口中的主机指纹，再清理 `known_hosts` 旧记录。

本节成果

- 镜像来源、文件名、版本和校验记录
- 完整烧录日志与首次启动记录
- RevyOS、内核、用户、网络、时间和 apt 软件源检查表
- 串口接线与终端参数记录
- SSH 服务状态、板端 IP 和主机指纹记录
- SSH 密钥登录验证 + SCP 双向传输与哈希校验结果

1.3 ruyi device provision 适用边界

学习目标

- 解释 device provision 的目的、输入、输出和潜在风险
- 使用当前 `ruyi` 版本查询设备支持信息
- 对比手工烧录流程（1.2）与自动化 provision 的适用场景与风险差异
- 根据设备、镜像、连接方式和数据风险选择自动 provision 或手工准备流程

provision 是什么

Device provision 是把设备准备成可使用状态的自动化过程，可能包括下载镜像、选择设备变体、写入存储介质或生成操作指引。它的价值是减少重复步骤，但自动化不消除风险——「能运行命令」不等于「适合运行命令」。

手工流程 vs 自动化 provision：六维对比

维度	手工流程（1.2）	自动化 provision
控制粒度	每步可控，可随时停止	流程封装，中断后需恢复
适用条件	只要有镜像和烧录工具即可	设备必须在软件源中明确列出
误操作风险	人为确认每一步，可发现异常	自动化写入可能跳过确认
可调试性	高，每步可单独验证	低，失败后需拆解自动化步骤
重复效率	低，需重复执行相同命令	高，一条命令完成多步
建议场景	首次使用、设备未列出、数据风险高	设备明确支持、重复部署、空白介质

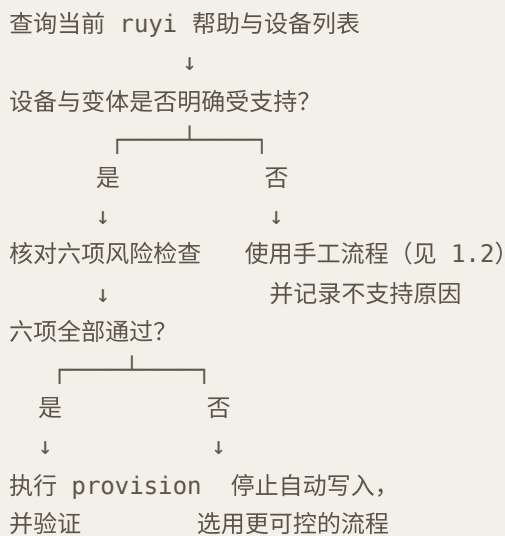
六项风险检查

在运行任何写入操作前逐项确认：

1. 设备是否在当前软件源中明确列出？
2. 变体是否与实物一致？
3. 写入目标是否唯一且可确认？
4. 重要数据是否已备份？
5. 供电和数据连接是否稳定？
6. 是否已准备串口或其他恢复通道？

关键原则：六项检查全部确认后，方可执行自动 provision。在不安全条件下应选用手工流程——课程接受任何有依据的决策路径。提交「当前版本不支持该设备」并描述手工替代方案同样是合格的实验结论。

决策流程



课堂练习

给出三个场景并作决策，说明选择自动或手工流程的理由：

1. 设备完全匹配且有重要数据
2. 设备未列出但名称相似
3. 设备列出且为新空白存储

本节成果

- 当前 `ruyi` 版本的 device/provision 帮助记录
- 课程目标板设备支持查询结果
- provision 风险检查表
- 自动或手工准备路径的决策记录与验证结果